

claude-windows / c7679509-ee2b-4443-b240-0b3b1b1a7291

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c7679509-ee2b-4443-b240-0b3b1b1a7291\subagents\agent-a919b4e427b4a7016.jsonl`  
- SHA-256: `389f2f7fcce28a71f3d633ec268918c1740e9d4ca075172605ba598e1e6c22e7`  
- Source modified: `2026-06-21T18:06:41+00:00`  
- Imported at: `2026-07-05T16:48:23+00:00`  
- project: `subagents`  
- session_id: `c7679509-ee2b-4443-b240-0b3b1b1a7291`
```

Transcript

1. user (2026-06-21T18:05:19.312Z)

Adversarially review PR P2b (PACKAGING_PLAN.md) in the git worktree C:/Users/avidu/Projects/bhi-wt-p2b. It adds a friendly `indexer --app` launcher: boot the local server (localhost-only) + open the bundled SPA in the browser.

Read in that worktree:

```
- backend/main.py: the new helpers `_port_is_free`, `_find_free_port`, `_resolve_app_port`, `_open_browser_when_ready` (search those names); the new `--app` arg; the CLI-branch guard (`args.video_path` and not `args.server` and not `args.app`); and the server-start block (search "args.app or args.server").  
- tests/test_app_launcher.py
```

Already verified: full suite 1302 passed/7 skipped, ruff+mypy clean, coverage 85.07%, and the subprocess test boots `indexer --app` (RALLY_NO_BROWSER=1) and confirms /api/health + the bundled SPA at /.

Adversarially check, with file:line evidence:

1. MODE INTERACTIONS: enumerate the combinations ? `--app`, `--server`, `--app --server`, `--video-path` alone (CLI single-shot), `--video-path --app`, no args (default?server). Does `--app` correctly (a) NEVER run CLI single-shot processing, (b) always run the server, (c) take precedence sensibly? Any combination that does something surprising or runs two modes?
2. LOCALHOST INVARIANT: `--app` forces bind 127.0.0.1 regardless of `security.edge\_auth\_enabled` / `--host` / `RALLY\_BIND\_HOST` (so the friendly app never exposes the network) ? confirm. Is that the intended/safe choice, and is it consistent with P0c (server still honors the profile)?
3. BROWSER-OPEN THREAD: `\_open\_browser\_when\_ready` runs in a daemon thread polling /api/health then webbrowser.open. Can it (a) hang the process / block Ctrl+C (daemon=True?), (b) open a dead page if the server fails to start, (c) leak, (d) race so the browser opens before the server is ready? Is RALLY_NO_BROWSER honored? Does the health poll's Host header pass the P0c TrustedHostMiddleware (127.0.0.1)?
4. FREE-PORT: `\_resolve\_app\_port` checks `\_port\_is\_free` then uvicorn binds later (TOCTOU). Is the window acceptable (best-effort + loud uvicorn failure)? Could `\_find\_free\_port` return a port that's then immediately rebindable? Does `--app` use the resolved port consistently for BOTH the browser URL and uvicorn.run?
5. SINGLE-INSTANCE (P0e) INTERACTION: --app still goes through the db lock + fail_stale_jobs (acquired before this block)? A 2nd `--app` on the same output-dir ? refuses (good)?
6. Any resource/socket leak in the helpers (sockets closed via `with`)? webbrowser.open exceptions swallowed?

Return ONLY: findings as "[SEVERITY blocker|major|minor|nit] <file:line> ? <issue> ? <fix>", then a one-line verdict. Be precise; don't invent issues. If clean, say so.

2. assistant (2026-06-21T18:05:22.124Z)

I'll review PR P2b adversarially. Let me start by reading the key files in the worktree.

3. user (2026-06-21T18:05:23.492Z)

```
4a06b8b feat(pkg): P1b ? bundle the KhelSutra SPA single-origin into the engine wheel (default-OFF) (#302)
be0c434 feat(pkg): P1a ? net-new console/wizard endpoints: /api/version, /api/capabilities, /api/videos, /api/reels (#300)
cb7394b feat(serving): T11 ? retention sweep for stale uploads + caches (#299)
c039b90 feat(pkg): P0e ? single-instance lock so a 2nd process can't corrupt in-flight job state (default-OFF) (#298)
e8bad40 feat(pkg): P0d ? declare scipy (undeclared serving dep) + license-scan & built-wheel CI lanes (#297)
---
On branch feat/packaging-p2b-launcher
Your branch is up to date with 'origin/master'.
```

```
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   backend/main.py
```

```
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        tests/test_app_launcher.py
```

no changes added to commit (use "git add" and/or "git commit -a")

4. user (2026-06-21T18:05:23.584Z)

```
1      """The friendly `indexer --app` launcher (PACKAGING_PLAN.md P2)."""
2
3      import os
4      import socket
5      import subprocess
6      import sys
7      import time
8      import urllib.error
9      import urllib.request
10
11     import pytest
12
13     import backend.main as m
14
15     REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
16
17
18     def _free_port() -> int:
19         s = socket.socket()
20         s.bind(("127.0.0.1", 0))
21         p = int(s.getsockname()[1])
22         s.close()
23         return p
24
25
26     # ---- port helpers ----
27
28     def test_find_and_check_free_port():
29         p = m._find_free_port("127.0.0.1")
30         assert isinstance(p, int) and p > 0
31         assert m._port_is_free("127.0.0.1", p) is True
32
33
34     def test_resolve_app_port_keeps_free_port():
35         p = _free_port()
36         assert m._resolve_app_port("127.0.0.1", p) == p
37
```

```

38
39 def test_resolve_app_port_falls_back_when_busy():
40     s = socket.socket()
41     s.bind(("127.0.0.1", 0))
42     s.listen()
43     busy = int(s.getsockname()[1])
44     try:
45         assert m._port_is_free("127.0.0.1", busy) is False
46         alt = m._resolve_app_port("127.0.0.1", busy)
47         assert alt != busy
48         assert m._port_is_free("127.0.0.1", alt) is True
49     finally:
50         s.close()
51
52
53 # ---- open-browser-when-ready ----
54
55 class _FakeResp:
56     status = 200
57     def __enter__(self): return self
58     def __exit__(self, *a): return False
59
60
61 def test_open_browser_opens_once_healthy(monkeypatch):
62     monkeypatch.delenv("RALLY_NO_BROWSER", raising=False)
63     monkeypatch.setattr(urllib.request, "urlopen", lambda *a, **k: _FakeResp())
64     opened = {}
65     monkeypatch.setattr(m.webbrowser, "open", lambda u: opened.setdefault("url", u) or
True)
66     assert m._open_browser_when_ready("http://127.0.0.1:1/",
"http://127.0.0.1:1/api/health", timeout=2) is True
67     assert opened["url"] == "http://127.0.0.1:1/"
68
69
70 def test_open_browser_suppressed_by_env(monkeypatch):
71     monkeypatch.setenv("RALLY_NO_BROWSER", "1")
72     monkeypatch.setattr(m.webbrowser, "open", lambda u: pytest.fail("must not open when
suppressed"))
73     assert m._open_browser_when_ready("http://x/", "http://x/api/health") is False
74
75
76 def test_open_browser_gives_up_when_never_ready(monkeypatch):
77     monkeypatch.delenv("RALLY_NO_BROWSER", raising=False)
78
79     def _boom(*a, **k):
80         raise OSError("connection refused")
81
82     monkeypatch.setattr(urllib.request, "urlopen", _boom)
83     monkeypatch.setattr(m.webbrowser, "open", lambda u: pytest.fail("must not open a
dead page"))
84     assert m._open_browser_when_ready("http://x/", "http://x/api/health", timeout=0.4)
is False
85
86
87 # ---- end-to-end: `indexer --app` boots the server + serves the bundled UI ----
88
89 def test_app_mode_boots_and_serves_bundled_ui(tmp_path):
90     """`indexer --app` (browser suppressed) must start the server localhost-only and
serve both
91     /api/health and the bundled SPA at /."""
92     port = _free_port()
93     env = {
94         **os.environ,
95         "RALLY_NO_BROWSER": "1", # don't pop a browser in CI
96         "PYTHONPATH": REPO_ROOT + os.pathsep + os.environ.get("PYTHONPATH", ""),

```

```

97         }
98         log = open(tmp_path / "app.log", "wb")
99         proc = subprocess.Popen(
100             [sys.executable, "-m", "backend.main", "--app", "--port", str(port),
101              "--output-dir", str(tmp_path / "out")],
102             cwd=REPO_ROOT, env=env, stdout=log, stderr=subprocess.STDOUT,
103         )
104         base = f"http://127.0.0.1:{port}"
105         try:
106             deadline = time.time() + 90
107             healthy = False
108             while time.time() < deadline:
109                 if proc.poll() is not None:
110                     pytest.fail(f"--app server exited early (code {proc.returncode}); see
{tmp_path/'app.log'}")
111                 try:
112                     with urllib.request.urlopen(f"{base}/api/health", timeout=2) as r:
113                         if r.status == 200:
114                             healthy = True
115                             break
116                     except (urllib.error.URLError, ConnectionError, OSError):
117                         time.sleep(0.5)
118                 assert healthy, "the --app server never became healthy"
119                 with urllib.request.urlopen(f"{base}/", timeout=5) as r:
120                     body = r.read().decode("utf-8", "replace")
121                 assert "/assets/" in body # the bundled SPA index.html (single-origin asset
refs)
122             finally:
123                 proc.terminate()
124                 try:
125                     proc.wait(timeout=10)
126                 except subprocess.TimeoutExpired:
127                     proc.kill()
128                 log.close()
129

```

5. assistant (2026-06-21T18:05:26.186Z)

Now let me read the relevant parts of `backend/main.py`. Let me find the helpers and the diff.

6. user (2026-06-21T18:05:27.503Z)

<persisted-output>

Output too large (50.1KB). Full output saved to: C:\Users\avidu\claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c7679509-ee2b-4443-b240-0b3b1b1a7291\tool-results\bfwzh3voj.txt

Preview (first 2KB):

```

diff --git a/backend/main.py b/backend/main.py
index 0f48c7e..6e95912 100644
--- a/backend/main.py
+++ b/backend/main.py
@@ -1,15 +1,25 @@
import os
import sys
import argparse
+import json
import logging
+import shutil
+import socket
import threading
-import traceback
import uuid
-from concurrent.futures import ThreadPoolExecutor
+import webbrowser

```

```

+from importlib.metadata import PackageNotFoundError, version as _pkg_version
+from urllib.parse import quote
import uvicorn
from dotenv import load_dotenv

-load_dotenv()
+## PACKAGING_PLAN.md P0a: prefer an app-home .env ONLY when RALLY_APP_HOME is set (the packaged
+## launcher pins it); otherwise env_file_to_load() returns None and load_dotenv(None) keeps the
+## exact historical frame-relative walk-up ? byte-identical to the pre-P0a bare load_dotenv().
+## stdlib-only import ? keeps this pre-pydantic line light.
+from backend.utils.app_home import env_file_to_load, resolve_output_dir
+
+load_dotenv(env_file_to_load())

# Centralized logging setup for the indexer (hot paths use logger, CLI output uses print for
direct user-facing messages).
logging.basicConfig(
@@ -20,6 +30,7 @@ logging.basicConfig(
    logger = logging.getLogger(__name__)
    from fastapi import FastAPI, HTTPException, Request
    from fastapi.middleware.cors import CORSMiddleware
+from starlette.middleware.trustedhost import TrustedHostMiddleware
    from pydantic import BaseModel
    from typing import List, Dict, Any, Optional

@@ -31,51 +42,209 @@ from backend.pipeline.segmenters.base import segmenter_registry
    from backend.pipeline.stitcher.compiler import VideoCompiler
    from backend.utils.hashing import compute_video_id # canonical file-identity hashing
    from backend.utils.paths import resolve_under_root, safe_output_filename, validate_input_path
-from backend.config import load_config as load_app_config, AppConfig, StitchingConfig # new
typed config
+from backend.utils.redact import redact_secrets # P0b: mask secret-shaped fields in
/api/config
...
</persisted-output>

```

7. assistant (2026-06-21T18:05:30.032Z)

The diff is large. Let me read the actual current `backend/main.py` to see the helpers and CLI logic in context with line numbers.

8. user (2026-06-21T18:05:32.063Z)

```

46:from backend.utils.single_instance import acquire_lock # P0e: one instance per database
114:     """P0c: the server bind address. Precedence: ``--host`` > ``RALLY_BIND_HOST`` env >
``0.0.0.0``
120:     env = os.environ.get("RALLY_BIND_HOST")
131:def _port_is_free(host: str, port: int) -> bool:
142:def _find_free_port(host: str) -> int:
149:def _resolve_app_port(host: str, port: int) -> int:
151:     return port if _port_is_free(host, port) else _find_free_port(host)
154:def _open_browser_when_ready(url: str, health_url: str, timeout: float = 60.0) -> bool:
156:     Returns True if a browser open was attempted. No-op (returns False) when
RALLY_NO_BROWSER is set
158:     if os.environ.get("RALLY_NO_BROWSER"):
226:# P0e: the single-instance lock handle. Kept module-global for the process lifetime ? if it
were
227:# GC'd/closed the OS would release the lock. None until main() acquires it (server/CLI
init).
449:     local_video = storage.acquire_local_input(req.video_path, getattr(global_config,
"storage", None))
751:     local_src = storage.acquire_local_input(video["filepath"], getattr(global_config,
"storage", None))
850:     parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
851:     parser.add_argument("--output-dir", default="output", help="Directory where processed

```

```

clips/highlights are saved")
852:     parser.add_argument("--setup", action="store_true", help="Launch dependency checker and
setup utility")
853:     parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a video
clip to run detailed diagnostics on")
854:     parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
855:     parser.add_argument("--app", action="store_true",
858:     parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI
server on")
859:     parser.add_argument("--host", default=None,
862:     parser.add_argument("--max-frames", type=int, help="Limit processing to the first N sub-
sampled frames for debugging")
865:     parser.add_argument("--segmenter", choices=available_segmenters, help=f"Choose rally
detection segmenter. Available: {available_segmenters}")
866:     parser.add_argument("--trim-start", type=float, help="Start time in seconds for the
debug trim segment feature")
867:     parser.add_argument("--trim-end", type=float, help="End time in seconds for the debug
trim segment feature")
868:     parser.add_argument("--trim-output", help="Output filename for the debug trimmed segment
(saves in output-dir unless absolute path)")
946:     _instance_lock = acquire_lock(db_path + ".lock")
950:     # legitimate single instance).
951:     logger.warning("[JOBS] Could not create the single-instance lock at %s.lock (%s);
proceeding "
965:     swept = db_instance.fail_stale_jobs()
970:     if args.video_path and not args.server and not args.app:
984:         local_video = storage.acquire_local_input(args.video_path, storage_cfg)
1051:     if args.app or args.server or not args.video_path:
1052:         if args.app:
1056:             port = _resolve_app_port(bind_host, args.port)
1061:             f"(set RALLY_NO_BROWSER=1 to skip). Press Ctrl+C to stop.")
1062:             threading.Thread(target=_open_browser_when_ready, args=(url, url +
"api/health"),

```

9. assistant (2026-06-21T18:05:34.104Z)

Now let me read the helper functions and the CLI/server-start blocks in detail.

10. user (2026-06-21T18:05:35.089Z)

```

110     )
111
112
113     def _resolve_bind_host(config: Optional[AppConfig] = None, cli_host: Optional[str] =
None) -> str:
114         """P0c: the server bind address. Precedence: ``--host`` > ``RALLY_BIND_HOST`` env >
``0.0.0.0``
115         when an edge-auth profile is active (``security.edge_auth_enabled`` ? i.e. there IS
an auth gate
116         in front) > ``127.0.0.1`` (default: localhost-only, today's behaviour). Binding a
routable
117         interface with no auth gate is unsafe, hence the localhost default; widen it
consciously."""
118         if cli_host and cli_host.strip():
119             return cli_host.strip()
120         env = os.environ.get("RALLY_BIND_HOST")
121         if env and env.strip():
122             return env.strip()
123         sec = getattr(config, "security", None)
124         if getattr(sec, "edge_auth_enabled", False):
125             return "0.0.0.0" # behind a Cloudflare-Access-style edge gate (M9) ? bind all
interfaces
126         return "127.0.0.1"
127

```

```

128
129     # --- P2: the friendly `indexer --app` launcher (cross-platform, no native component ?
01) ---
130
131     def _port_is_free(host: str, port: int) -> bool:
132         """True if (host, port) can be bound right now (best-effort; a TOCTOU race is
acceptable ?
133         uvicorn surfaces a real bind failure loudly)."""
134         with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
135             try:
136                 s.bind((host, port))
137                 return True
138             except OSError:
139                 return False
140
141
142     def _find_free_port(host: str) -> int:
143         """An OS-assigned free port on host."""
144         with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
145             s.bind((host, 0))
146             return int(s.getsockname()[1])
147
148
149     def _resolve_app_port(host: str, port: int) -> int:
150         """For --app: keep `port` if free, else fall back to a free one so a busy 8000 never
blocks the app."""
151         return port if _port_is_free(host, port) else _find_free_port(host)
152
153
154     def _open_browser_when_ready(url: str, health_url: str, timeout: float = 60.0) -> bool:
155         """Poll `health_url` until the server is up, then open `url` in the user's default
browser.
156         Returns True if a browser open was attempted. No-op (returns False) when
RALLY_NO_BROWSER is set
157         (tests / headless) or the server never becomes healthy (don't open a dead page).
Never raises."""
158         if os.environ.get("RALLY_NO_BROWSER"):
159             return False
160         import time as _time
161         import urllib.request
162         deadline = _time.monotonic() + timeout
163         while _time.monotonic() < deadline:
164             try:
165                 with urllib.request.urlopen(health_url, timeout=2) as r:
166                     if getattr(r, "status", 200) == 200:
167                         break
168             except Exception:
169                 _time.sleep(0.3)
170         else:
171             return False
172         try:
173             webbrowser.open(url)
174         except Exception:
175             pass
176         return True
177
178
179     # Serves static frontend files directly (now under api/ per approved structure).
180     # PACKAGING_PLAN.md P0a: resolve PACKAGE-relative (not CWD-relative) so launching from
an
181     # arbitrary directory finds the shipped assets instead of creating a stray
./backend/api/static.
182     # (The legacy in-engine dashboard; the real SPA gets bundled here in P1a.)
183     _STATIC_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "api", "static")
184     try:

```

```

185         os.makedirs(_STATIC_DIR, exist_ok=True)
186     except OSError:
187         # The dir ships with the package (a no-op create); guard against a read-only install
188         # tree (system-installed Python) since exist_ok suppresses FileExistsError, not
189         # PermissionError.
190         pass
191     app.mount("/static", StaticFiles(directory=_STATIC_DIR), name="static")
192     # P1b: serve the bundled KhelSutra SPA single-origin. `backend/ui/` is the committed
193     # snapshot
194     # (produced by scripts/bundle_ui.sh); fall back to the legacy in-engine dashboard only
195     # if it's
196     # absent (a source checkout with no vendored UI). The SPA calls same-origin `/api`, so
197     # no SPA change
198     # is needed; it is query-param driven (?video=?), so serving index.html at `/' + /assets
199     # is enough
200     # (no client-side path routing ? no history-fallback catch-all, which would also be a
201     # route-order hazard).
202     _BUNDLED_UI_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "ui")
203     _UI_DIR = _BUNDLED_UI_DIR if os.path.isfile(os.path.join(_BUNDLED_UI_DIR, "index.html"))
204     else _STATIC_DIR
205     _UI_ASSETS_DIR = os.path.join(_UI_DIR, "assets")
206     if os.path.isdir(_UI_ASSETS_DIR):
207         app.mount("/assets", StaticFiles(directory=_UI_ASSETS_DIR), name="assets")
208
209     @app.get("/", response_class=HTMLResponse)
210     def serve_index():
211         index_path = os.path.join(_UI_DIR, "index.html")
212         if os.path.exists(index_path):
213             with open(index_path, "r", encoding="utf-8") as f:
214                 return f.read()
215         return "<h3>Sports Highlight Indexer Server is Running. (No bundled UI found.)</h3>"
216
217     def _bundled_ui_version() -> Optional[Dict[str, Any]]:
218         """Provenance of the vendored SPA (backend/ui/ui_version.json), or None if no UI is
219         bundled.
220         Includes `engine_api_version` ? the API_VERSION the snapshot was built against (skew
221         guard)."""
222         path = os.path.join(_BUNDLED_UI_DIR, "ui_version.json")
223         try:
224             with open(path, "r", encoding="utf-8") as f:
225                 return json.load(f)

```

11. assistant (2026-06-21T18:05:37.813Z)

Now let me look at the TrustedHostMiddleware setup (P0c) and the server-start block.

12. user (2026-06-21T18:05:39.874Z)

```

33:from starlette.middleware.trustedhost import TrustedHostMiddleware
77:app.add_middleware(
86:def _allowed_hosts_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:
97:    # IPv4 loopback (the default bind) + the TestClient host. NB: Starlette's
98:    # TrustedHostMiddleware
99:    TrustedHostMiddleware(
100:        TrustedHostMiddleware,
101:        allowed_hosts=_allowed_hosts_from_env(),

```

13. user (2026-06-21T18:05:40.712Z)

```

920         print("[ERROR] Please provide --video-path to debug a specific clip.")
921         return

```

```

922         run_cli_diagnose_clip(args.video_path, args.debug_clip)
923         return
924
925     # Initialize Global Config and DB
926     global global_config, db_instance
927     global_config = load_app_config() # returns typed AppConfig
928     _enforce_startup_or_exit(global_config) # M5: fail fast on an incoherent ACTIVE
profile
929
930     # The CLI --output-dir overrides the configured output directory. It now lives
931     # on the typed model (OutputConfig.output_dir), so every consumer ? including
932     # /api/compile ? reads the same value instead of a write-only runtime hack.
933     # P0a: a RELATIVE output dir roots under the app-home (RALLY_APP_HOME); with the env
934     # unset, resolve_output_dir("output") == "output" (CWD-relative) ? byte-identical to
before.
935     global_config.output.output_dir = resolve_output_dir(args.output_dir)
936     os.makedirs(global_config.output.output_dir, exist_ok=True)
937
938     db_path = os.path.join(global_config.output.output_dir,
global_config.output.db_name)
939
940     # P0e: refuse a SECOND instance against the same DB BEFORE the destructive stale-job
sweep below.
941     # The job worker is in-process, so a concurrent process opening this DB would sweep
THIS one's
942     # in-flight jobs to 'failed'. The OS lock (held for the process lifetime; auto-
released on exit
943     # or crash) is keyed to db_path, so a different --output-dir/DB still runs
concurrently.
944     global _instance_lock
945     try:
946         _instance_lock = acquire_lock(db_path + ".lock")
947     except OSError as exc:
948         # The lock path isn't writable (read-only dir / perms). Don't crash and don't
falsely claim a
949         # second instance ? warn and proceed WITHOUT the guard (a lock-infra failure
must not block a
950         # legitimate single instance).
951         logger.warning("[JOBS] Could not create the single-instance lock at %s.lock
(%s); proceeding "
952                        "WITHOUT the second-instance guard.", db_path, exc)
953         _instance_lock = None
954     else:
955         if _instance_lock is None:
956             print(f"[FATAL] Another instance is already using this database ({db_path}).
Refusing to "
957                  f"start ? a second instance would corrupt in-flight job state. Stop
the other one, or "
958                  f"use a different --output-dir (RALLY_APP_HOME) to run an independent
instance.")
959             sys.exit(1)
960
961     db_instance = Database(db_path)
962
963     # Crash recovery (#16): jobs left queued/running by a previous process are dead ?
964     # the executor is in-process. Mark them failed so pollers see a terminal state.
965     swept = db_instance.fail_stale_jobs()
966     if swept:
967         logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ?
failed.")
968
969     # CLI processing mode (no web UI needed). --app/--server always run the server,
never CLI single-shot.
970     if args.video_path and not args.server and not args.app:
971         print(f"[CLI] Processing video file: {args.video_path}")

```

```

972         # M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is
973         fetched to scratch.
974         # The local-existence fast-fail only applies to a local input (stat the RESOLVED
975         key, not the
976         # raw URI); a remote fetch fails loudly. An unsupported scheme is a clean error,
977         not a crash.
978         storage_cfg = getattr(global_config, "storage", None)
979         try:
980             input_ref = storage.resolve_input_ref(args.video_path, storage_cfg)
981         except ValueError as e:
982             print(f"[FAIL] {e}")
983             return
984         if input_ref.backend == "local" and not os.path.exists(input_ref.key):
985             print(f"[FAIL] Video file not found: {args.video_path}")
986             return
987         local_video = storage.acquire_local_input(args.video_path, storage_cfg)
988
989         video_id = compute_video_id(local_video)
990         was_reingest = db_instance.get_video(video_id) is not None
991
992         # Validate (with-context variant surfaces ffprobe_meta for the report)
993         print("[CLI] Validating...")
994         val_results, val_context = registry.run_all_with_context(local_video,
995         global_config)
996         failures = [r for r in val_results if not r.passed]
997
998         # Save video status
999         status = "failed" if failures else "processed"
1000         db_instance.add_video(video_id, args.video_path, status, [r.model_dump() for r
1001         in val_results])
1002
1003         seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
1004         "default_segmenter", "motion")
1005         segmenter_actual = None
1006         segmentation_ran = False
1007         try:
1008             print("\n" + "="*40)
1009             print("VALIDATION SUMMARY")
1010             print("="*40)
1011             for r in val_results:
1012                 status_symbol = "[PASS]" if r.passed else "[FAIL]"
1013                 print(f"{status_symbol} {r.validator_name}: {r.message}")
1014             print("="*40 + "\n")
1015
1016             if failures:
1017                 print("[FAIL] Video failed checks. Indexing halted.")
1018                 return
1019
1020             # Index
1021             segmenter_cls = segmenter_registry.get(seg_name)
1022             if not segmenter_cls:
1023                 print(f"[FAIL] Segmenter '{seg_name}' not found.")
1024                 return
1025
1026             print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
1027             segmenter_actual = seg_name
1028             play_wm, cand_wm = db_instance.segment_watermarks(video_id)
1029             segmenter = segmenter_cls(db_instance, global_config)
1030             result = segmenter.process_video(video_id, local_video,
1031             max_frames=args.max_frames)
1032             segmentation_ran = True
1033
1034             if isinstance(result, Failure):
1035                 print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
1036                 print("[FAIL] Indexing halted for video.")

```

```

1030         segments = []
1031     else:
1032         segments = result.value if hasattr(result, "value") else result
1033         print(f"[SUCCESS] Process complete. Indexed {len(segments)} play
segments inside database: {db_path}")
1034         # Destroy-AFTER-confirm: keep new on success, restore prior on
empty/failure.
1035         _finalize_reingest(video_id, segments, play_wm, cand_wm)
1036     finally:
1037         # Always emit the per-video observability report. Never raises.
1038         paths = emit_indexer_report(
1039             db=db_instance, video_id=video_id, video_path=args.video_path,
config=global_config,
1040             val_results=val_results, val_context=val_context,
1041             segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
1042             was_reingest=was_reingest, segmentation_ran=segmentation_ran,
1043         )
1044         if paths:
1045             print(f"[REPORT] Technical report : {paths.get('technical_md')}")
1046             print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
1047             print(f"[REPORT] Machine JSON      : {paths.get('json')}")
1048         return
1049
1050     # Start Local Server Mode
1051     if args.app or args.server or not args.video_path:
1052         if args.app:
1053             # P2: the friendly launcher ? always localhost-only (use --server for
network exposure),
1054             # auto-pick a free port if --port is busy, and open the bundled UI once the
server is up.
1055             bind_host = "127.0.0.1"
1056             port = _resolve_app_port(bind_host, args.port)
1057             if port != args.port:
1058                 print(f"[APP] Port {args.port} is busy ? using {port} instead.")
1059             url = f"http://{bind_host}:{port}/"
1060             print(f"[APP] Starting KhelSutra ? opening {url} in your browser "
f"(set RALLY_NO_BROWSER=1 to skip). Press Ctrl+C to stop.")
1061             threading.Thread(target=_open_browser_when_ready, args=(url, url +
"api/health"),
1062                             daemon=True).start()
1063         else:
1064             bind_host = _resolve_bind_host(global_config, args.host) # P0c:
config/profile-driven (default 127.0.0.1)
1065             port = args.port
1066             print(f"[SERVER] Starting local FastAPI server on
http://{bind_host}:{port}...")
1067             if bind_host not in ("127.0.0.1", "localhost", "::1"):
1068                 print(f"[SERVER] ? Binding {bind_host} exposes the API on the network ?
ensure an auth "
1069                       f"gate (edge_auth_enabled / a reverse proxy) is in front, and set
"
1070                       f"RALLY_ALLOWED_HOSTS to your public host(s) (the Host-header
guard defaults to "
1071                       f"loopback-only and will 400 forwarded requests otherwise).")
1072             # Resume self-scheduled work orphaned by the restart (EXECUTION_POLICY.md P3):
the
1073             # in-process wake-timer for any parked 'waiting' job died with the old process,
so
1074             # kick one drain ? it reclaims due jobs and re-arms the wake-timer for the rest.
1075             _get_executor().submit(_drain_due_jobs)
1076             uvicorn.run(app, host=bind_host, port=port)
1077
1078 if __name__ == "__main__":

```

Now let me read the TrustedHostMiddleware / allowed-hosts block and the health endpoint to verify the Host-header invariant.

15. user (2026-06-21T18:05:45.559Z)

```
77     app.add_middleware(  
78         CORSMiddleware,  
79         allow_origins=_cors_origins_from_env(),  
80         allow_credentials=False,  
81         allow_methods=["*"],  
82         allow_headers=["*"],  
83     )  
84  
85  
86     def _allowed_hosts_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:  
87         """P0c: Host-header allowlist ? a DNS-rebinding guard for the localhost appliance. A  
malicious  
88         web page can point a domain it controls at 127.0.0.1 and script requests to the  
local server;  
89         the browser sends ``Host: attacker.com``, which this rejects (the server only  
answers loopback  
90         Host headers by default). ``RALLY_ALLOWED_HOSTS`` (comma-separated) overrides; the  
default keeps  
91         Starlette's TestClient host ``testserver`` so the in-process suite isn't broken.  
Read from the  
92         ENV (not config.json) so importing ``backend.main`` does zero filesystem I/O; a  
routable/edge  
93         deploy sets its public host (or ``*`` when fronted by an auth gate that validates  
the host)."""  
94         src = os.environ if env is None else env  
95         raw = (src.get("RALLY_ALLOWED_HOSTS", "") or "").strip()  
96         hosts = [h.strip() for h in raw.split(",") if h.strip()]  
97         # IPv4 loopback (the default bind) + the TestClient host. NB: Starlette's  
TrustedHostMiddleware  
98         # matches on host.split(":")[0], which mangles a bracketed IPv6 Host ("::1:port"),  
so IPv6  
99         # loopback is NOT supported by this guard ? access the appliance via  
127.0.0.1/localhost.  
100         return hosts or ["localhost", "127.0.0.1", "testserver"]  
101  
102  
103     # DNS-rebinding guard. Bound to localhost by default, so only loopback Host headers are  
honoured  
104     # (a remote page can't read responses cross-origin AND its Host header is rejected  
here). www_redirect  
105     # off so an unexpected ``www.`` host is rejected rather than 307-redirected.  
106     app.add_middleware(  
107         TrustedHostMiddleware,  
108         allowed_hosts=_allowed_hosts_from_env(),  
109         www_redirect=False,  
110     )  
111  
112
```

16. user (2026-06-21T18:05:46.951Z)

```
backend/main.py:298:@app.get("/api/health")  
backend/main.py:299:def health():  
backend/main.py:1062:         threading.Thread(target=_open_browser_when_ready, args=(url,  
url + "api/health"),
```

17. assistant (2026-06-21T18:05:51.831Z)

Now I have the full picture for the Host-header concern. The browser-open thread polls `url +`

"api/health" where `url = f"http://{bind\_host}:{port}/" and `bind_host = "127.0.0.1". So `urllib.request.urlopen` will send `Host: 127.0.0.1:<port>`, which is in the default allowlist. Good.

Let me verify a few edge cases. First, the `url + "api/health"` concatenation? `url` ends with `/`, so it becomes `http://127.0.0.1:port/api/health`. Correct. Let me double-check the `--setup` branch and the early-return branches that come before the lock acquisition (to confirm `--app` always reaches the lock + fail_stale_jobs).

18. user (2026-06-21T18:05:52.574Z)

```
870     args = parser.parse_args()
871
872     if args.setup:
873         # Invoke the diagnostics/bootstrap script (renamed from the old root
874         # setup.py to scripts/doctor.py so it no longer shadows the PEP 517
875         # build system). Resolve its path from this file so `indexer --setup`
876         # works regardless of the caller's cwd.
877         import subprocess
878         doctor_path = os.path.join(
879             os.path.dirname(os.path.abspath(__file__)),
880             "scripts", "doctor.py",
881         )
882         print(f"[INFO] Invoking diagnostics: {doctor_path} ...")
883         subprocess.run([sys.executable, doctor_path])
884         return
885
886     if args.trim_start is not None and args.trim_end is not None:
887         if not args.video_path:
888             print("[ERROR] Please provide --video-path to extract a segment.")
889             return
890
891         # Determine output path
892         out_filename = args.trim_output or "trimmed_segment.mp4"
893         if os.path.isabs(out_filename):
894             out_path = out_filename
895             out_dir = os.path.dirname(out_path)
896             out_name = os.path.basename(out_path)
897         else:
898             out_dir = resolve_output_dir(args.output_dir) # P0a: app-home aware (CWD-
identical when unset)
899             out_path = os.path.join(out_dir, out_filename)
900             out_name = out_filename
901
902         os.makedirs(out_dir, exist_ok=True)
903         print(f"[CLI] Extracting debug segment from {args.trim_start}s to
{args.trim_end}s from {args.video_path}...")
904
905         # global_config isn't initialized this early; load the typed config so the trim
906         # clip carries the same configured stitching paddings + watermark as a real
907         # highlight (watermark default-on).
908         stitching = load_app_config().stitching
909         compiler = VideoCompiler(out_dir, begin_padding=stitching.begin_padding,
910                                 end_padding=stitching.end_padding,
watermark=stitching.watermark)
911         try:
912             res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
args.trim_end)], out_name)
913             print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
914         except Exception as e:
915             print(f"[ERROR] Failed to compile trimmed segment: {e}")
916         return
917
918     if args.debug_clip is not None:
919         if not args.video_path:
```

```

920             print("[ERROR] Please provide --video-path to debug a specific clip.")
921             return
922         run_cli_diagnose_clip(args.video_path, args.debug_clip)
923         return
924

```

19. assistant (2026-06-21T18:05:56.237Z)

Now I have the complete picture. Let me check the argparse `--app` and `--host` help text definitions and also verify whether there's any issue with `--app --video-path` combined with the early `--debug-clip`/`--trim` branches (which return before reaching the server block ? but those require explicit flags, so they're independent of `--app`).

20. user (2026-06-21T18:05:57.100Z)

```

850         parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
851         parser.add_argument("--output-dir", default="output", help="Directory where
processed clips/highlights are saved")
852         parser.add_argument("--setup", action="store_true", help="Launch dependency checker
and setup utility")
853         parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a
video clip to run detailed diagnostics on")
854         parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
855         parser.add_argument("--app", action="store_true",
856                             help="Launch the app: start the local server (localhost-only)
and open the "
857                                "bundled UI in your browser. Falls back to a free port if
--port is busy.")
858         parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI
server on")
859         parser.add_argument("--host", default=None,
860                             help="Bind address. Default 127.0.0.1 (localhost-only); 0.0.0.0
when "
861                                "security.edge_auth_enabled. Expose on a network ONLY
behind an auth gate.")
862         parser.add_argument("--max-frames", type=int, help="Limit processing to the first N
sub-sampled frames for debugging")
863

```

21. assistant (2026-06-21T18:06:02.975Z)

I now have everything needed. Let me trace the mode-interaction matrix carefully and verify the `--app --host` interaction (does `--app` truly ignore `--host`?).

Looking at lines 1051-1063: when `args.app` is True, `bind\_host = "127.0.0.1"` is hardcoded ? `args.host` and `\_resolve\_bind\_host` are completely bypassed. So `--app --host 0.0.0.0` still binds 127.0.0.1. Confirmed.

Let me verify one subtle point about `\_open\_browser\_when\_ready`'s `else` clause on the while loop, and check the `\_port\_is\_free` `SO\_REUSEADDR` concern for the TOCTOU window. Let me also confirm `acquire\_lock` semantics and whether `--app` reaches it. Let me check there are no other early returns between line 924 and 944.

22. user (2026-06-21T18:06:04.573Z)

```

4:     global_config = load_app_config() # returns typed AppConfig

```

23. user (2026-06-21T18:06:06.663Z)

```

---uvicorn---
571:         sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

```

24. assistant (2026-06-21T18:06:17.279Z)

There's a subtle TOCTOU nuance worth noting: `\_port\_is\_free` binds without `SO\_REUSEADDR`, while uvicorn binds *with* `SO\_REUSEADDR=1`. This is fine for the "is it free" check (a strictly-free port passes both). But there's an asymmetry worth flagging in `\_find\_free\_port`: it returns an OS-assigned ephemeral port, which is genuinely free at that instant. The window is acceptable per the documented intent.

Let me confirm one last thing: the `\_open\_browser\_when\_ready` thread's behavior on a server that fails to bind (uvicorn raises before serving). The thread will poll health until timeout (60s) then return False without opening ? good, no dead page. But the main thread's `uvicorn.run` would raise and crash the process; since the browser thread is daemon, it dies with the process. No hang. Confirmed.

Let me verify the test's claim about coverage of the `--app --server` combination and check whether the daemon thread could open a browser AFTER the process is meant to exit (e.g. fast Ctrl+C). The daemon thread is killed at interpreter exit, so no late browser-open after shutdown.

I have enough to write the findings. Let me do one final check ? whether `webbrowser.open` could *block* (some platforms block waiting for the browser). It runs in a daemon thread, so even if it blocks, it won't prevent process exit. Confirmed safe.

Let me verify the health-poll Host header concern definitively by checking what `urllib` sends and that `127.0.0.1:port` passes the allowlist (TrustedHostMiddleware strips the port via `host.split(":")[0]` ? `127.0.0.1`, which is in the default list).

25. user (2026-06-21T18:06:18.176Z)

```
298 @app.get("/api/health")
299 def health():
300     """Cheap liveness/readiness probe (no auth) for the SPA's offline banner
301     (HOSTING_PLAN.md:42). Reports the configured sport + default segmenter so the UI can
302     confirm the engine is reachable and which detector is wired. Never raises."""
303     sport = getattr(global_config, "sport", None)
304     indexing = getattr(global_config, "indexing", None)
305     return {
```

26. assistant (2026-06-21T18:06:41.520Z)

All confirmed. The health endpoint is auth-free and the poll uses `Host: 127.0.0.1:<port>` which passes the TrustedHostMiddleware default allowlist. Note one interaction worth flagging: if a user sets `RALLY\_ALLOWED\_HOSTS` without including `127.0.0.1`, the health poll would 400 ? but the default keeps `127.0.0.1`, and `--app` forces 127.0.0.1 bind, so the default path is safe. Setting `RALLY\_ALLOWED\_HOSTS` to an exclusive non-loopback set while using `--app` is a self-inflicted misconfiguration; the health poll would then never see 200 and the browser simply wouldn't open (no dead page). Acceptable.

Here are my findings.

****Adversarial review ? PR P2b (`indexer --app` launcher)****

****1. Mode interactions**** ? Clean. The CLI single-shot guard at `backend/main.py:970` (`if args.video\_path and not args.server and not args.app`) correctly excludes `--app`, so `--video-path --app` never runs single-shot processing and falls through to the server block at `:1051` (`if args.app or args.server or not args.video\_path`). `--app --server` ? `--app` branch wins (`:1052` checks `args.app` first) ? localhost-only, which is the safe precedence. No-args ? server (`not args.video\_path`). No combination runs two modes. The `--setup/--trim/--debug-clip` branches (`:872`,`:886`,`:918`) return early and require their own flags, independent of `--app`.

****2. Localhost invariant**** ? Confirmed. `:1055` hardcodes `bind\_host = "127.0.0.1"` for `--app`, fully bypassing `\_resolve\_bind\_host` / `--host` / `RALLY\_BIND\_HOST` / `edge\_auth\_enabled`. `--server` still honors the P0c profile via `\_resolve\_bind\_host(global\_config, args.host)` at `:1065`. Intended and safe ? the friendly app can never expose the network.

****3. Browser-open thread**** ? Solid. Daemon thread (``:1062-1063` `daemon=True`) ? won't block process exit or Ctrl+C. `RALLY\_NO\_BROWSER` honored at ``:158`. Dead-page guard: the `while/else` at ``:163-171` returns False without opening if health never returns 200 (no open on a server that fails to bind). `webbrowser.open` is wrapped in try/except (``:172-175`), exceptions swallowed. No race: it polls `/api/health` and only opens after a 200. Health URL is `url + "api/health"` = `http://127.0.0.1:<port>/api/health` (url ends in `/`, ``:1059`) ? `urllib` sends `Host: 127.0.0.1:<port>`, which passes TrustedHostMiddleware (port stripped ? `127.0.0.1`, in the default allowlist at ``:100`). `/api/health` is auth-free (``:298`).

****4. Free-port (TOCTOU)**** ? Acceptable as documented. `\_resolve\_app\_port` (``:149-151`) checks `\_port\_is\_free` then uvicorn binds later. `\_port\_is\_free` (``:131`) binds without `SO_REUSEADDR`; uvicorn binds with `SO_REUSEADDR=1` ? a strictly-free port passes both, so the check is not falsely positive. `_find_free_port` (``:142`) returns an OS-assigned ephemeral port (free at that instant). Window is best-effort; uvicorn surfaces a real bind failure loudly (docstring ``:132-133`). The resolved `port` is used consistently for BOTH the browser URL (``:1059`) and `uvicorn.run` (``:1077`).

****5. Single-instance (P0e)**** ? Correct. `--app` reaches `acquire\_lock(db\_path + ".lock")` at ``:946` and `fail\_stale\_jobs()` at ``:965` (no early return between ``:924` and the lock ? verified). A 2nd `--app` on the same `--output-dir` ? lock fails ? `[FATAL]` + `sys.exit(1)` at ``:956-959`. Lock-infra failure (read-only dir) warns and proceeds without the guard (``:951`), not a crash.

****6. Resource/socket leaks**** ? None. Both `\_port\_is\_free` (``:134`) and `\_find\_free\_port` (``:144`) use `with socket.socket(...)` ? sockets always closed. `urllib.request.urlopen` uses `with` (``:165`). `webbrowser.open` exceptions swallowed (``:174`).

****[nit] backend/main.py:95-100**** ? If a user sets `RALLY\_ALLOWED\_HOSTS` to an exclusively non-loopback set *and* uses `--app`, the health poll's `Host: 127.0.0.1` would 400 and the browser would silently never open (no dead page, no crash). Self-inflicted misconfiguration and the default path is safe; worth a one-line note that `--app` always needs loopback in the allowlist, but not blocking.

****Verdict:**** Clean ? no blocker/major/minor issues; one nit. Ship it.